UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, *Controller*, *Model*, dan *Middleware* pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Jassen Nova Tabalujan
NPM	2428240026
Kelas	SI5B
Topik Tugas 1	Rental Alat Kemah
Entitas utama	Camping Gear
Tautan repositori	https://github.com/jassengeren/si5b_kuis1_jassen_nova_tabalujan
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	8 Oktober

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]` dan `[Entitas]`, dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, `[sumber-daya]` menjadi `buku`.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan `POST`. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama `PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf`, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian *endpoint*	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau `curl` di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan `Win` + `Shift` + `S`; macOS tekan `Cmd` + `Shift` + `4`; Linux gunakan `PrtSc` atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda `[...]` pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Rental Alat Kemah
Entitas utama (tunggal)	Camping Gear
Alamat dasar	/camping-gears
Field data	id, namaAlat, kategori, kapasitas, hargaSewaPerHari, stok
Field wajib saat POST	namaAlat, kategori, kapasitas, hargaSewaPerHari, stok
Query string untuk filter	Tidak ada
Data awal	Tenda Dome 4 Orang, Tas Carrier 60L, Kompor Portable
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey`.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/camping-gears	Tidak	200	Menampilkan seluruh data camping gear
2	GET	/camping-gears/:id	Tidak	200	Menampilkan satu data berdasarkan ID
3	POST	/camping-gears	Ya	201	Menambahkan data camping gear
4	PUT	/camping-gears/:id	Ya	200	Mengubah data camping gear
5	DELETE	/camping-gears/:id	Ya	204	Menghapus data camping gear

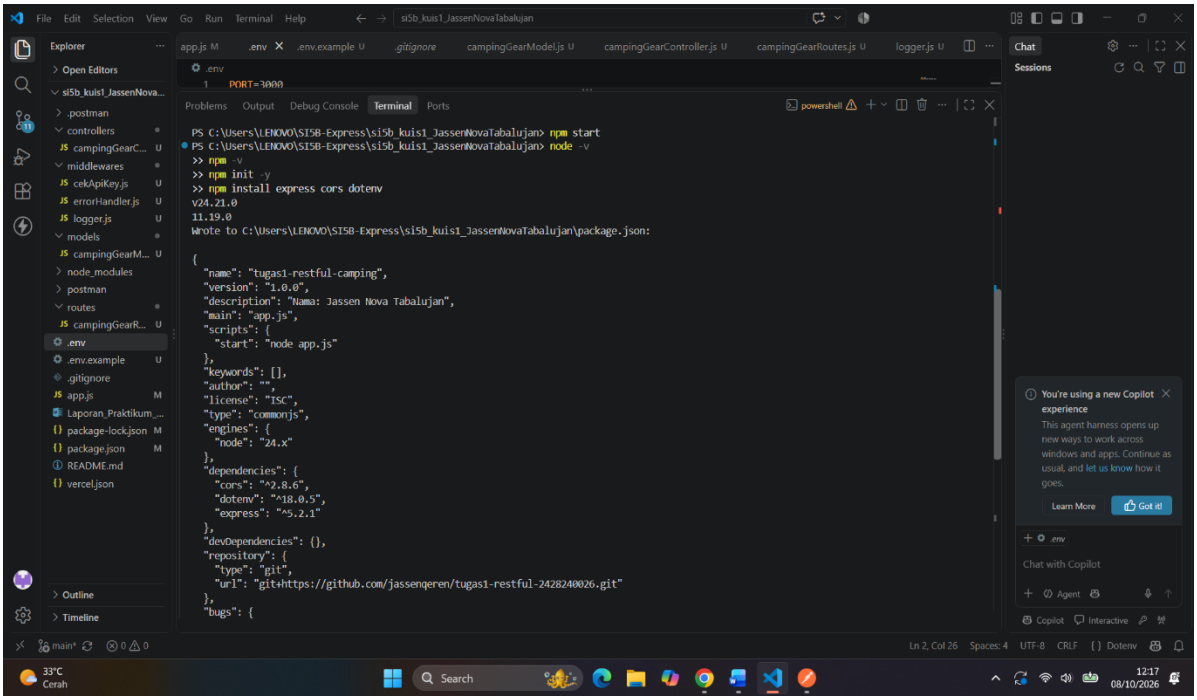
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

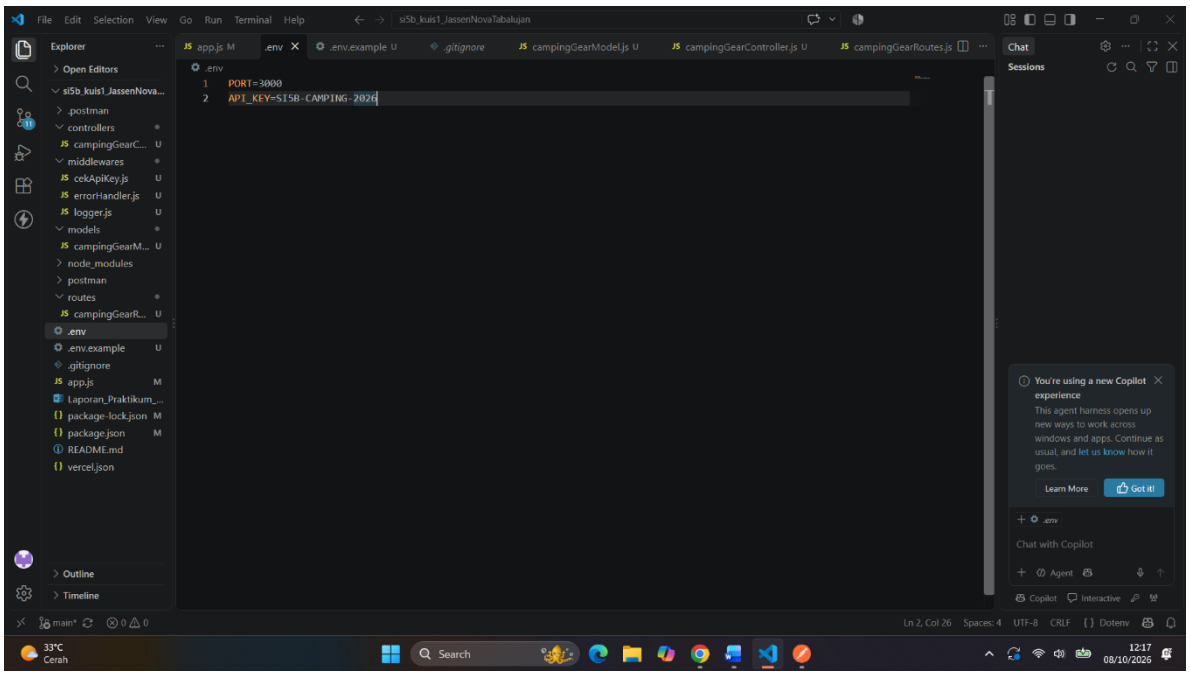
Pada project ini saya menggunakan Node.js sebagai runtime untuk menjalankan aplikasi backend. Express digunakan untuk membuat RESTful API dan mengatur route, CORS digunakan agar API dapat menerima request dari sumber atau aplikasi lain, sedangkan dotenv digunakan untuk membaca konfigurasi dari file .env. Instalasi dilakukan menggunakan npm melalui terminal.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder `routes`, `controllers`, `models`, dan `middlewares` beserta isinya. Gunakan panel *Explorer* editor atau perintah `tree`.



Gambar 2. Struktur folder project

Penjelasan:

Struktur project saya dipisahkan menjadi beberapa layer agar setiap bagian memiliki tanggung jawab yang berbeda. Folder models digunakan untuk menyimpan data dan fungsi pengolahan data, controllers menangani request dan response, routes mengatur alamat endpoint, sedangkan middlewares menangani proses tambahan seperti logger, API key, dan error handler.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

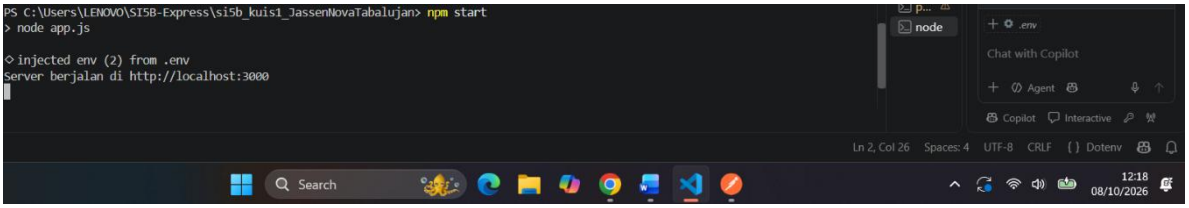
Berkas	Penjelasan Anda
models/campingGearModel.js	Menyimpan data camping gear serta fungsi untuk mengambil semua data, mencari berdasarkan ID, menambah, mengubah, dan menghapus data. Model tidak menggunakan req dan res.
controllers/campingGearController.js	Menangani request dari pengguna, melakukan validasi data, memanggil fungsi pada Model, kemudian memberikan response JSON dan status HTTP.

Berkas	Penjelasan Anda
routes/campingGearRoutes.js	Mendefinisikan endpoint /camping-gears menggunakan express.Router() dan menghubungkan setiap method HTTP dengan fungsi Controller.
middlewares/logger.js	Mencatat informasi request yang masuk ke server seperti method dan alamat endpoint.
middlewares/cekApiKey.js	Memeriksa API key dari header request dan melindungi endpoint POST, PUT, dan DELETE.
middlewares/errorHandler.js	Menangani error secara terpusat sehingga error seperti JSON tidak valid dan route yang tidak ditemukan dapat dikembalikan dalam format JSON.
app.js	Menjadi file utama aplikasi yang memuat dotenv, CORS, middleware JSON, route camping gear, logger, dan error handler serta menjalankan server.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Server dijalankan menggunakan perintah npm start. Nomor port berasal dari konfigurasi PORT yang terdapat pada file .env, yaitu port 3000. Jika port tersebut sedang digunakan oleh aplikasi lain, server tidak dapat menggunakan port yang sama sehingga perlu menghentikan aplikasi yang menggunakan port tersebut atau mengganti nilai port pada konfigurasi.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

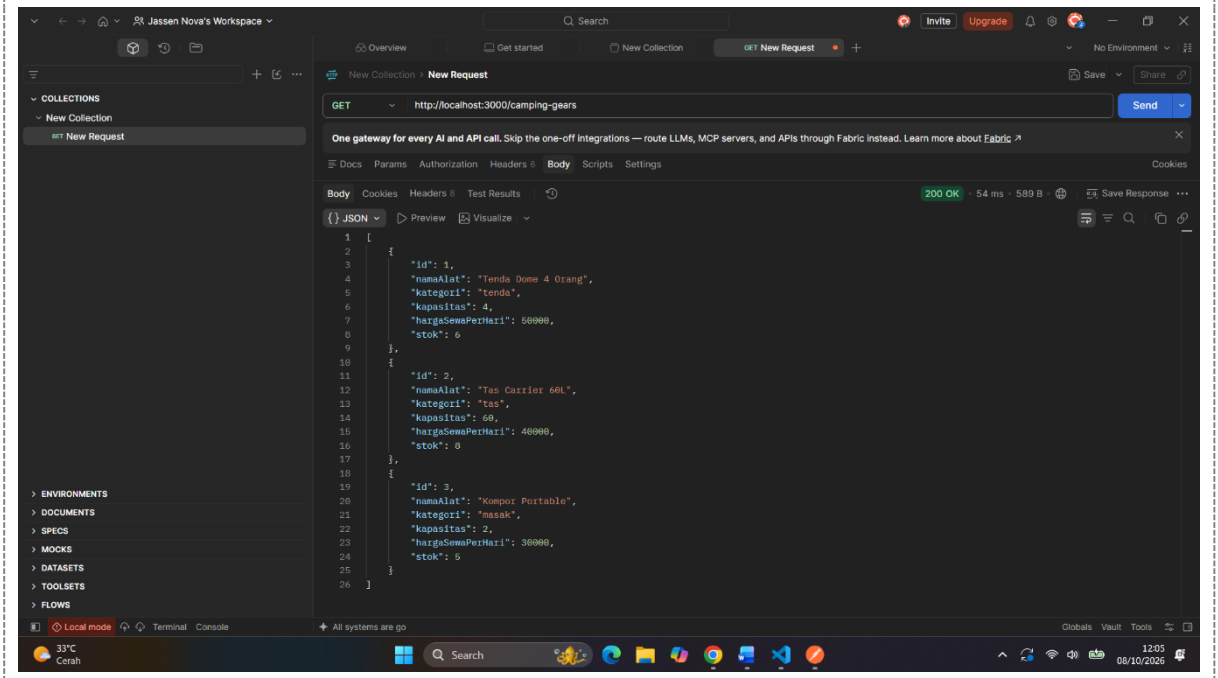
> **Catatan**
> Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".
> Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /camping-gears)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (*request*)	GET /camping-gears
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK
Lulus?	[✓] Ya [ ] Tidak



Gambar 4. Hasil GET semua data

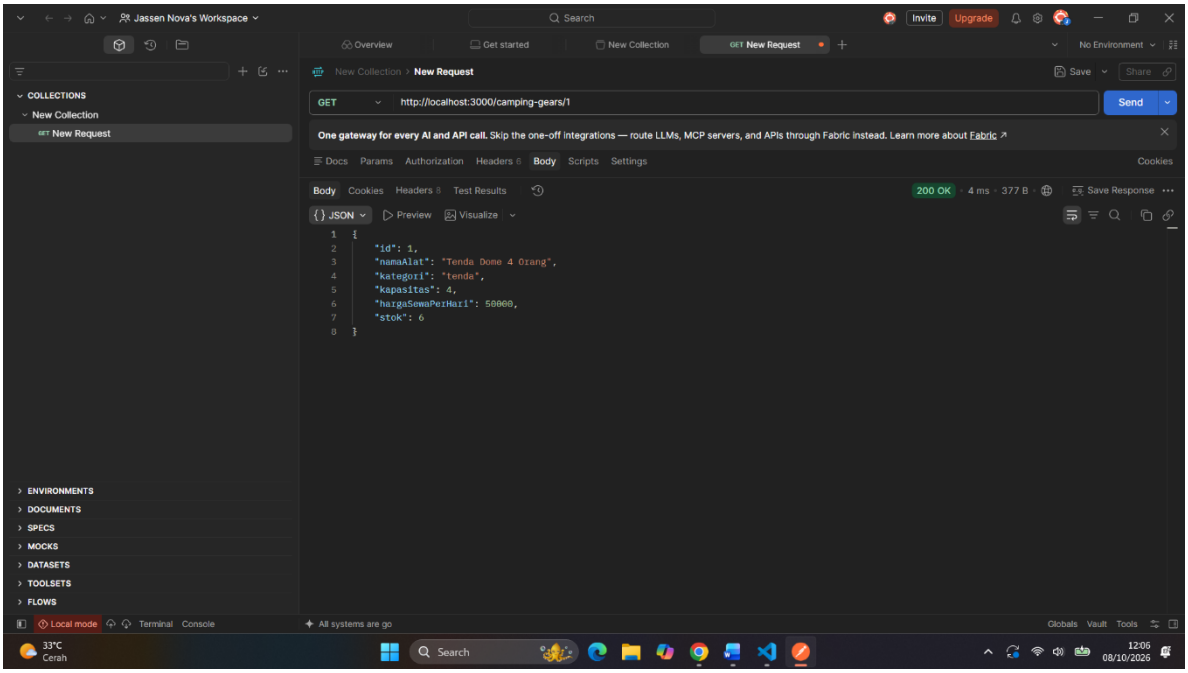
Penjelasan:

Endpoint GET /camping-gears berhasil mengembalikan seluruh data camping gear dengan status 200 OK. Request diterima oleh route, kemudian Controller memanggil fungsi Model untuk mengambil seluruh data dan hasilnya dikirimkan kembali dalam bentuk array JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (*request*)	GET /camping-gears/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK
Lulus?	[✓] Ya [ ] Tidak



Gambar 5. Hasil GET berdasarkan ID

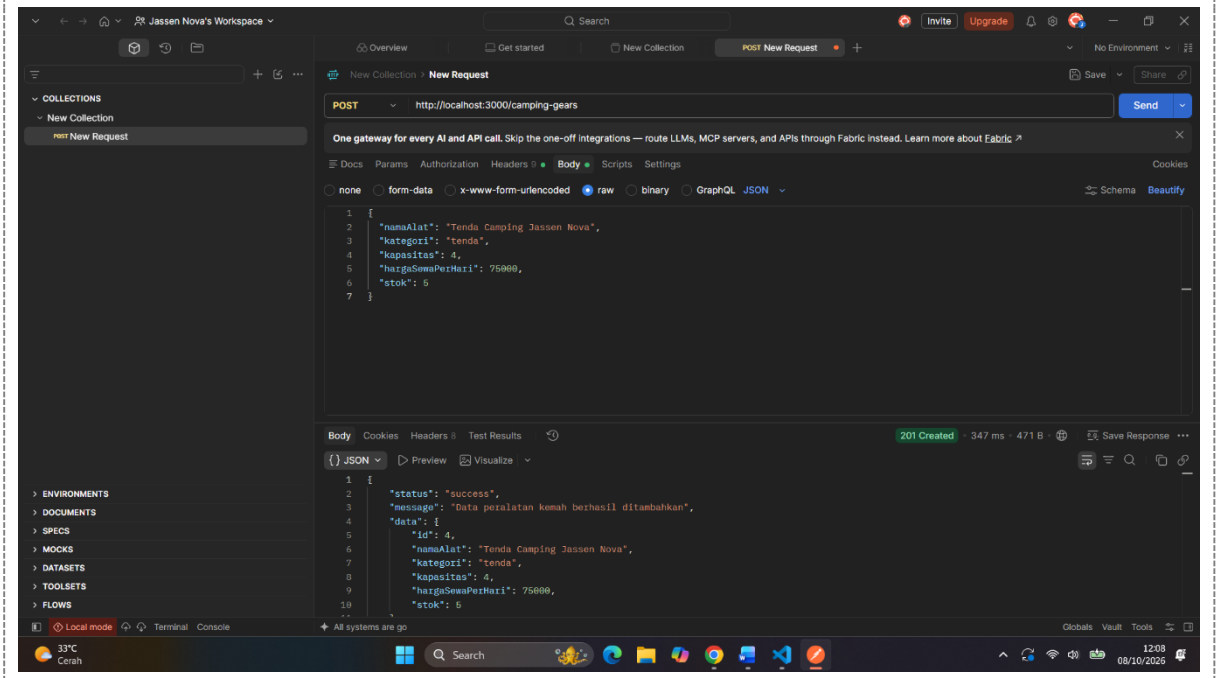
Penjelasan:

Endpoint mengambil data camping gear berdasarkan ID yang terdapat pada URL. Nilai ID diperoleh melalui req.params.id, kemudian diubah menjadi angka agar dapat dibandingkan dengan ID yang tersimpan pada Model. Karena ID 1 ditemukan, server mengembalikan satu objek camping gear dengan status 200 OK..

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (*request*)	POST /camping-gears
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field wajib 1]": "x-api-key", "[field wajib 2]": "SI5B-CAMPING-2026 " }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created
Lulus?	[✓] Ya [ ] Tidak



Gambar 6. Hasil POST data baru

Penjelasan:

Pada proses POST, middleware cekApiKey memeriksa API key yang dikirim melalui header x-api-key. Setelah API key valid, request diteruskan ke Controller. express.json() digunakan untuk membaca body JSON, kemudian Controller melakukan validasi dan memanggil Model untuk menambahkan data. Status 201 Created digunakan karena request berhasil membuat resource atau data baru.

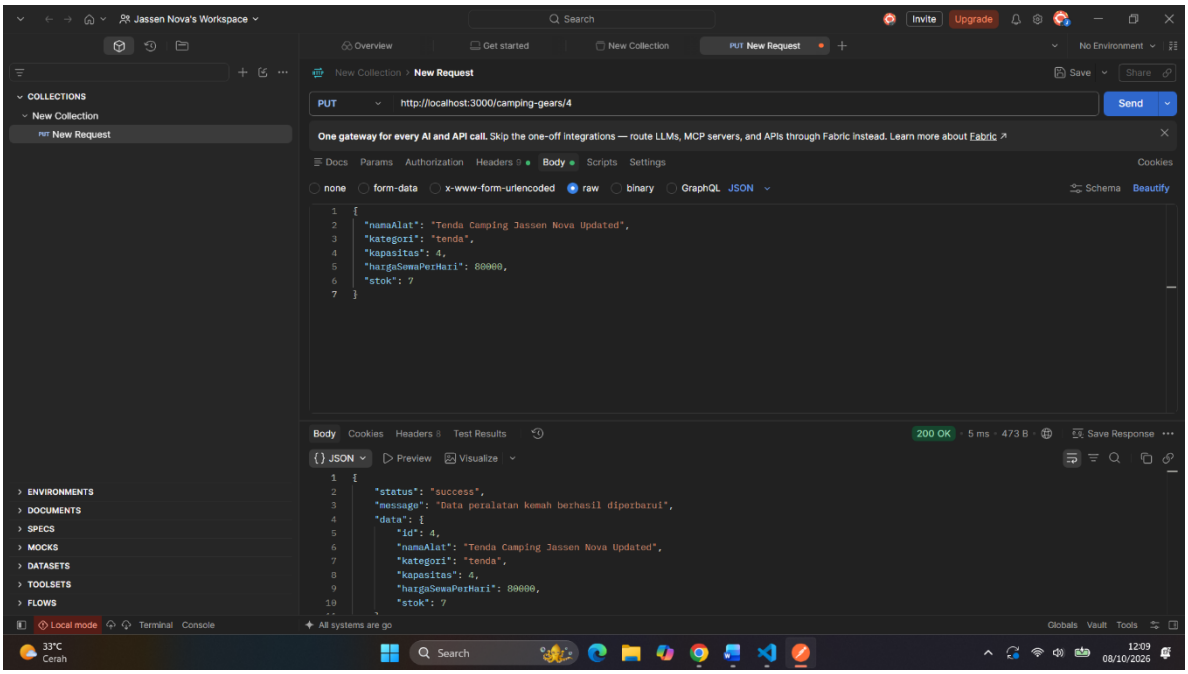
T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (*request*)	PUT /camping-gears/4
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json

Body	{
	"namaAlat": "Tenda Camping Jassen Nova Updated",
	"kategori": "tenda",
	"kapasitas": 4,
	"hargaSewaPerHari": 80000,
	"stok": 7
	}

Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK
Lulus?	[✓] Ya [ ] Tidak



Gambar 7. Hasil PUT mengubah data

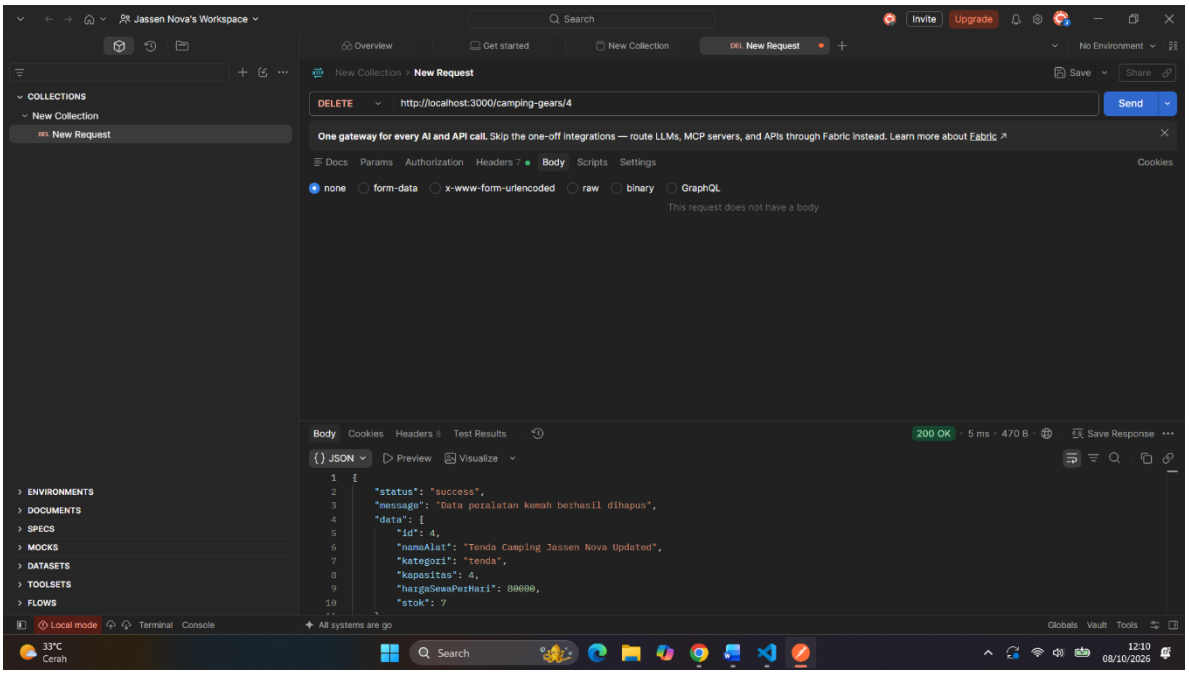
Penjelasan:

Endpoint PUT berhasil mengubah data camping gear dengan ID 4. Field yang berubah adalah namaAlat, hargaSewaPerHari, dan stok, sedangkan field id, kategori, dan kapasitas tetap. Proses perubahan dilakukan oleh fungsi Model setelah Controller menerima dan memvalidasi request.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (*request*)	DELETE /camping-gears/4
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	200 OK
Lulus?	[ ] Ya [✓] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Endpoint DELETE digunakan untuk menghapus data camping gear berdasarkan ID. Request dengan ID 4 berhasil diproses dan server mengembalikan status 200 OK sesuai dengan implementasi API saya. Data yang dihapus adalah data camping gear yang sebelumnya dibuat pada pengujian POST dan kemudian diperbarui melalui PUT.

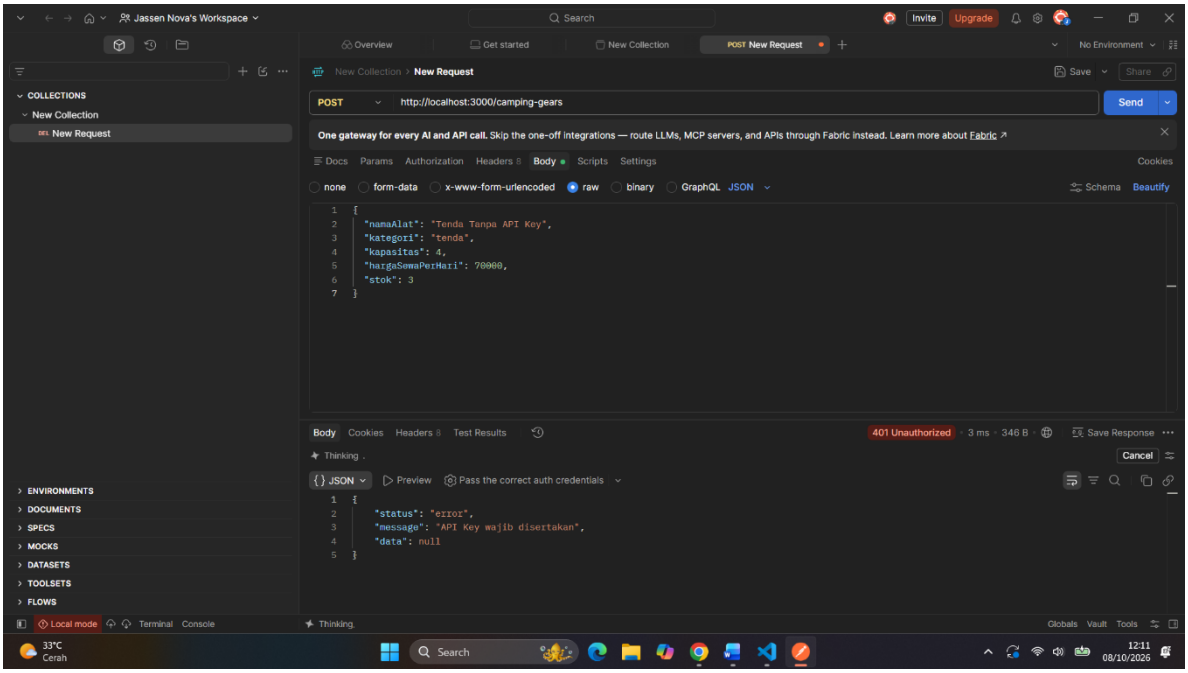
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (*request*)	POST /camping-gears
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized
Lulus?	[✓] Ya [ ] Tidak



Gambar 9. POST tanpa API key ditolak

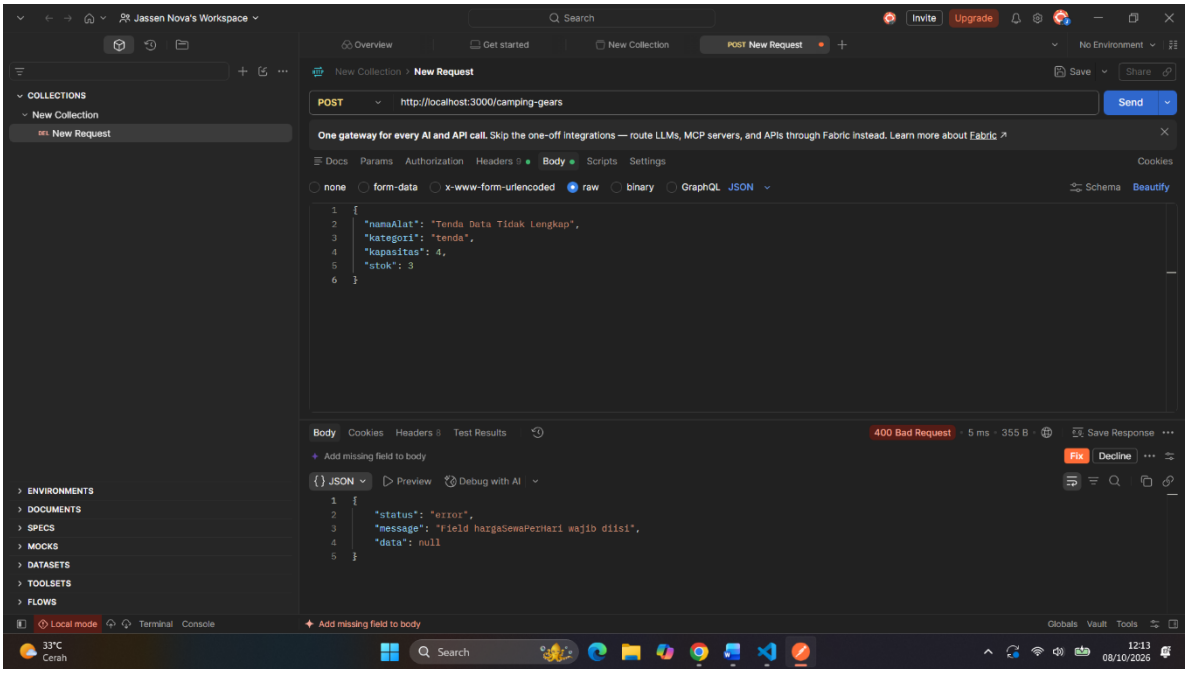
Penjelasan:

Request POST tanpa header x-api-key ditolak oleh middleware cekApiKey. Karena pemeriksaan API key dilakukan sebelum Controller dijalankan, Controller tidak memproses data tersebut. Server mengembalikan status 401 Unauthorized sebagai tanda bahwa request tidak memiliki kredensial API key yang diperlukan..

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (*request*)	POST /camping-gears
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ hanya sebagian field wajib }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request
Lulus?	[✓] Ya [ ] Tidak



Gambar 10. POST dengan body tidak lengkap

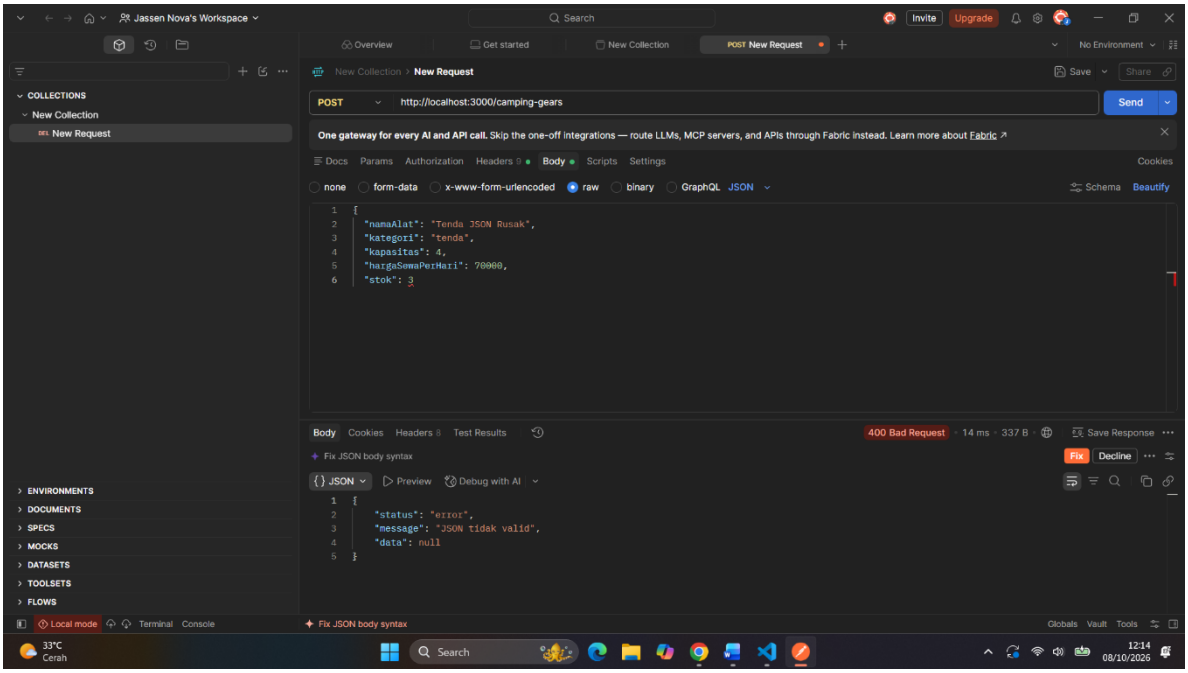
Penjelasan:

Request POST ditolak karena terdapat field wajib yang belum dikirim, yaitu hargaSewaPerHari. Validasi dilakukan pada Controller sebelum data diteruskan ke Model. Status 400 Bad Request digunakan karena kesalahan berasal dari data request yang dikirim oleh client.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (*request*)	POST /camping-gears
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request
Lulus?	[✓] Ya [ ] Tidak



Gambar 11. POST dengan JSON rusak

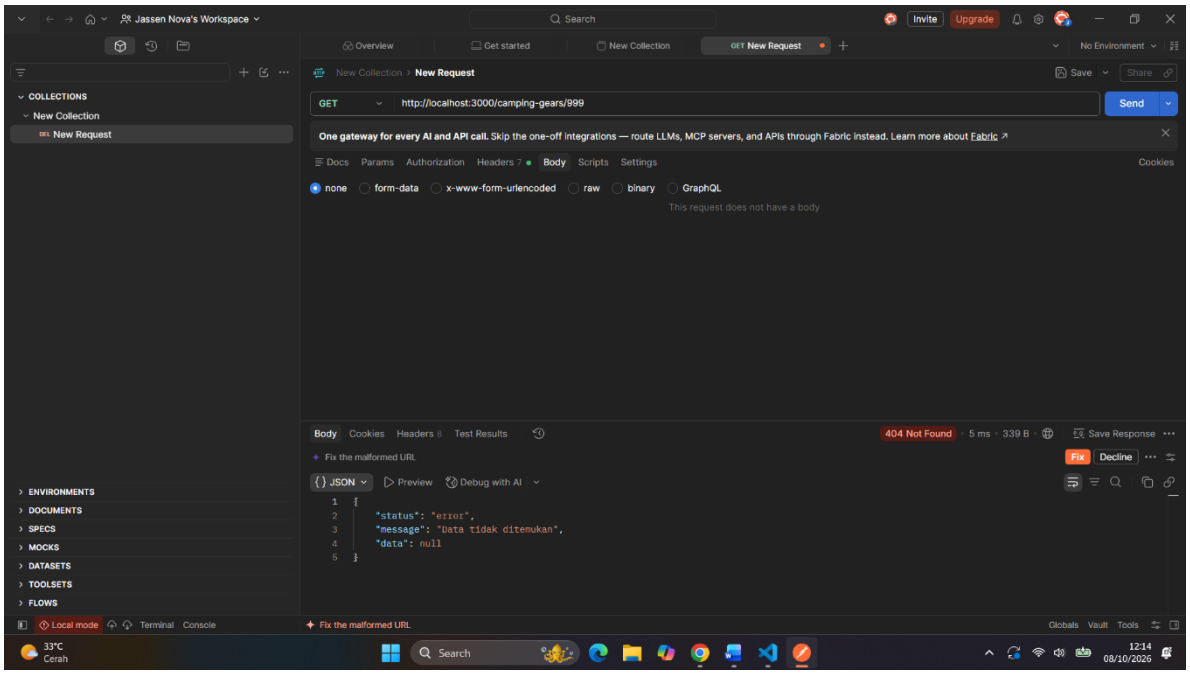
Penjelasan:

JSON yang dikirim tidak memiliki format yang valid sehingga tidak dapat diproses sebagai body request. Kesalahan tersebut ditangkap oleh middleware atau error handler setelah proses parsing JSON gagal. Error kemudian dikembalikan dalam bentuk response JSON dengan status 400 sehingga client menerima pesan kesalahan yang lebih terstruktur.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (*request*)	GET /camping-gears/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found
Lulus?	[✓] Ya [ ] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Request menggunakan ID 999 yang tidak terdapat pada data camping gear. Controller meminta Model mencari data tersebut, tetapi Model tidak menemukan objek dengan ID tersebut sehingga Controller mengembalikan error 404. Response kemudian diberikan dalam format JSON dengan pesan Data tidak ditemukan.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	ya
T02	Menampilkan satu data berdasarkan ID	200	200 OK	ya
T03	Menambah data baru	201	201 Created	ya
T04	Mengubah data	200	200 OK	ya
T05	Menghapus data	204	200 OK	tidak
N01	Menambah data tanpa API key	401	401 Unauthorized	ya
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	ya
N03	Mengirim JSON yang rusak	400	400 Bad Request	ya
N04	Mengambil ID yang tidak ada	404	404 Not Found	ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/jassenqeren/si5b_kuis1_jassen_nova_tabalujan
Pesan commit terakhir	Refactor API camping gear ke arsitektur backend terstruktur
Folder routes, controllers, models, middlewares ada	[☑] Ya [ ] Tidak
.env tidak terunggah (ada di .gitignore)	[☑] Ya [ ] Tidak
.env.example ada	[☑] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Pada project Camping Gear, Route bertugas menentukan alamat dan method endpoint lalu menghubungkannya ke Controller, misalnya router.get("/", campingGearController.getAllCampingGears). Controller menangani request dan response, misalnya fungsi getAllCampingGears yang memanggil Model dan mengirim hasilnya menggunakan res.json(). Model bertugas mengolah data camping gear, misalnya fungsi untuk mengambil seluruh data tanpa menggunakan req dan res..

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan kesalahan menjadi lebih konsisten sehingga setiap route tidak perlu menulis kode error yang sama berulang kali. Pada pengujian GET /camping-gears/999, ketika data tidak ditemukan, API dapat mengembalikan response 404 dalam format JSON yang sama melalui penanganan error terpusat.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan topik Tugas 1 saya	✓
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	✓
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	✓
4	Identitas saya (nama atau NIM) muncul pada data uji POST	✓
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	✓
6	Tabel rekapitulasi sesuai dengan tangkapan layar	✓
7	Tautan repositori pada sampul dapat dibuka dan `.env` tidak terunggah	✓
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	✓

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; `.env` tidak terunggah	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		**100**